TORSHIELD: A Transparent In-Kernel Authenticated Tunnel with Fail-Closed TC Steering

Roshan Srikanth
Amrita Vishwa Vidyapeetham
Email: roshansrikanth21@gmail.com

Yash Yashuday
Amrita Vishwa Vidyapeetham
Email: yyashuday@gmail.com

Dr. S. UdhayaKumar
Amrita Vishwa Vidyapeetham
Email: sudhayakumar@ch.amrita.edu

***Abstract*—TORSHIELD started as a testable question: can a Linux module encrypt only traffic bound for chosen peers, leave applications untouched, and fail closed when a TC classifier disappears? The module wraps the inner IP packet in UDP, protects a 16-byte header with ChaCha20–Poly1305, and keeps per-peer 64-bit sequence numbers, a 1024-packet replay window, and 64-byte padding. Netfilter, an L3 device, and a TC clsact program share the same encapsulation code. In a QEMU guest on kernels 6.8.0-147 and 6.17.0-42, 1 MB IPv4 and 256 KB IPv6 TCP transfers were byte-exact, an independent verifier opened 1054/1054 tunnel frames, and deleting the TC classifier blocked 8/8 local sends and 8/8 forwarded packets with zero outer frames. A 30 s four-stream run on a native WSL2 VM held 3.40 Gbit/s with flat unreclaimable memory. On its own, TORSHIELD protects payloads but does not hide who is talking. A further enhancement is to plug the outer tunnel into a Tor pluggable transport, so that it can borrow Tor's anonymity and censorship resistance. It has no handshake or forward secrecy yet.**

***Index Terms*—Linux kernel, network security, authenticated encryption, ChaCha20–Poly1305, Netfilter, eBPF, traffic control, replay protection.**

I. INTRODUCTION AND PROBLEM STATEMENT

We began with a small experiment. Take two network namespaces, route a chosen destination through a kernel module, and see whether an unmodified `iperf3` client can finish a transfer while a capture on the underlay shows only UDP frames. That experiment became TORSHIELD. The motivation is ordinary: many hosts still run software we cannot easily modify, and TLS leaves addresses, ports, packet sizes, and timing in the clear. A module below the socket can apply one policy to every application, including old binaries and test tools.

The first prototype used a single-byte XOR. We are not going to defend it. Known plaintext recovers the byte immediately, the length does not change, and there is no integrity check. The XOR code remains only as a baseline, and we do not use its numbers. Everything below refers to the AEAD datapath.

On its own, TORSHIELD protects payloads but does not hide who is talking: it does not pad to a fixed cell size, add cover traffic, or hide the outer addresses or the fact that two hosts are exchanging UDP. A future enhancement is to plug the outer tunnel into a Tor pluggable transport, which would add anonymity without changing the datapath. It is also not a WireGuard replacement: keys are configured by hand, there is no handshake, and there is no forward secrecy. What it does have is a kernel path that can be switched among Netfilter, an L3 route, and TC without changing the wire format, plus a guard that drops packets when the TC classifier is gone.

Three things came out of the work. First, we have a datapath and wire format for peer-destined IPv4/IPv6 traffic: ChaCha20–Poly1305, a 32-bit key identifier, 64-bit per-peer sequence numbers, 64-byte padding, a 1024-packet replay window, and per-peer allowed prefixes with authenticated roaming. Second, the same wire format is used by Netfilter, a route-based L3 device, and a TC clsact/eBPF classifier, and a kernel-side guard at `LOCAL_OUT`/`FORWARD` drops peer-destined packets when the classifier or qdisc is missing. Third, we have a measurement record from two kernel versions and a native VM, including independent AEAD verification, deletion tests, manual rekey, a rate limiter, and a 30 s run. That record is the baseline for the next round of work.

II. STATE OF THE ART

Tor is the reference point for onion routing, and its relay network is built to hide who is talking to whom [1]. Pluggable transports such as obfs4 and ScrambleSuit change the observable shape of a flow [2], [3]; format-transforming encryption makes ciphertext look like a chosen format [4]. Those systems solve a harder problem than ours. They try to hide that a flow is Tor, or make it resemble something else. TORSHIELD does neither. Traffic morphing changes sizes and timing [5], and Parrot showed that naive obfuscation can be fingerprinted [6]. Our 64-byte padding reduces exact-length leakage, but it does not make the traffic unobservable, and we do not describe it as obfuscation.

IPsec is the standards-based answer for authenticated and encrypted IP, with a policy and key-exchange framework behind it [7]. WireGuard is a small kernel UDP tunnel with a Noise-based handshake, cryptokey routing, and forward secrecy [8]. TORSHIELD borrows the UDP/AEAD shape from WireGuard but stops at a different point: no handshake, manually configured per-peer keys, and a transparent Netfilter path plus a TC/eBPF path. We do not claim better security or better throughput than WireGuard. For production privacy, WireGuard or IPsec is the right tool.

eBPF and TC let us classify and redirect packets inside the kernel [9]. We use a clsact egress program only for steering. The AEAD stays in `ndo_start_xmit` because

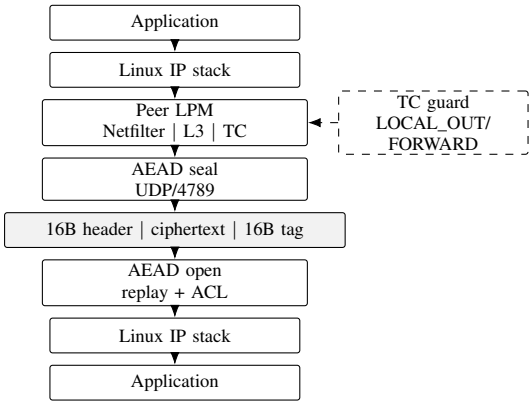


Fig. 1. The v4 datapath. The three egress modes feed the same AEAD code. The TC guard is a separate Netfilter hook and drops peer-destined packets when the classifier or qdisc disappears.

the BPF helper set cannot call the synchronous in-kernel ChaCha20/Poly1305 path. The guard is a separate Netfilter hook, and that separation creates the ordering problem described in Section III-B.

III. Proposed Approach

A. Wire Format and Modes

TorShield is a loadable module, and it does nothing until a key and at least one peer exist. Loading it therefore does not disturb a host. Fig. 1 shows the shared path. The default mode is `tunnel`. The `hybrid` and `tc` modes change only how egress packets reach the encapsulation code; they keep the same wire format and the same Netfilter ingress path. The legacy `stream` mode is length-preserving and is not recommended.

An outer packet is `outer IP | UDP:4789 | 16-byte header | padded ciphertext | 16-byte tag`. The 16-byte header is the AEAD associated data. It starts with the two-byte `TS` magic, then a version byte, a flags byte (bit 0 marks an inner IPv6 packet), a 32-bit key identifier, and a 64-bit big-endian per-peer sequence number. The nonce is the 4-byte XOR fold of the outer source and destination addresses followed by the 8-byte sequence number. The plaintext is the complete inner IP packet, zero-padded to a multiple of 64 bytes. The inner header carries the true length, and we parse it only after authentication. ChaCha20–Poly1305 follows RFC 8439 [10]. The outer IPv4 identification field comes from a per-packet counter, so fragments from different datagrams remain distinguishable at reassembly.

A peer entry holds a 32-byte key, a deterministic 32-bit key identifier (full FNV-1a of the key, with zero reserved), an outer endpoint, and up to eight allowed inner prefixes. Egress picks the peer by longest-prefix match on the inner destination. Ingress looks up the peer by the wire key identifier, authenticates, and then checks the decrypted inner source against that peer's prefixes. A packet with a source outside the allowed set is dropped and counted. The key identifier is a selector, not an authenticator: an observer can read it, but cannot make a valid tag without the key. After authentication, a peer with a per-peer key may roam to a new outer source address.

B. Steering and the Fail-Closed Guard

In `tunnel` mode, Netfilter `POST_ROUTING` matches the inner destination against peer prefixes, encapsulates, and returns `NF_STOLEN`. In `hybrid` mode, a route into a `torshield` L3 netdevice is encapsulated by `ndo_start_xmit`; other peer traffic still uses Netfilter. In `tc` mode, a clsact eBPF classifier on the underlay redirects matching inner packets to the `torshield` device. Outer frames carry mark `0x5454` so they are not redirected twice. If a peer matches but the classifier has no redirect target, the program returns `TC_ACT_SHOT`, not `TC_ACT_OK`. A matching packet must never fall through to the wire.

Classifier removal is the hard case. Netfilter `POST_ROUTING` runs before TC egress. In TC mode the Netfilter egress hook is bypassed, otherwise packets would be encapsulated twice. Relying on the classifier alone would therefore be fail-open if the qdisc or program disappeared. TorShield registers IPv4/IPv6 `NF_INET_LOCAL_OUT` and `NF_INET_FORWARD` hooks at the first priority, both before `POST_ROUTING`. For a peer-destined packet, the guard uses `nf_hook_state.out`, not `skb_dst`, because `skb_dst` can still describe the input route at `FORWARD`. It accepts only if the output device is a `torshield` device, or if the device still has a live TC egress classifier. Otherwise it returns `NF_DROP` and increments `tc_guard`. A local `sendto()` then fails with `EPERM`; no clear frame reaches the underlay. The TCX branch is compile-tested only; the runtime tests use the clsact path.

The guard proves that a classifier is attached. It does not prove that the classifier's maps are complete. `torshield-tc attach` reads every `allowed=` prefix from `/proc/torshield/peers` and fills the LPM tries before switching the mode to `tc`. If a peer is added, deleted, or cleared while the mode is `tc`, the kernel forces the mode back to `tunnel` until the loader reruns. Direct map edits or a hand-loaded classifier are outside the guarantee. The guard and the later TC egress hook read the attachment separately, so a classifier deleted after the guard accepts a packet can still let that one in-flight packet leave in clear. The deletion tests prove fail-closed behavior for subsequent packets, not a zero-race guarantee.

C. Replay, Rate Limiting, and Rekey

The receiver keeps a 1024-packet sliding window per peer. Sequence zero is invalid. The window moves only after successful authentication, so an off-path packet cannot consume a slot. An optional per-source fixed-window limiter runs before `skb_linearize` and Poly1305. It bounds the crypto work one source can force, but it is not a cookie challenge and the source address is spoofable. Per-peer keys can be rotated by hand: `rekey-peer` moves the current key to a previous

TABLE I
REPRESENTATIVE CAPTURED RESULTS. EVERY VALUE COMES FROM A RUN, NOT A PROJECTION.

Check	Captured result
Tunnel TCP	1 MB IPv4 and 256 KB IPv6 byte-exact
AEAD verification	1054/1054 tunnel; 807/807 hybrid v4; 209/209 hybrid v6; 210/210 TC v4; 68/68 TC v6
Wrong-key control	0/210 TC v4; 0/68 TC v6
`LOCAL_OUT` delete	0/8 sent, server 0, `tc_guard` +8, `dev_out` +0
`FORWARD` delete	`n3` outer 0, `tc_guard` +8, `dev_out` +0
30 s native run	3.40 Gbit/s, four streams, `SUnreclaim` 63–68 MB, 0 drops/replays

slot and starts sending with the new key; the receiver accepts both key identifiers until `rekey-commit` erases the previous key. A second rotation before commit is rejected. There is no handshake and no forward secrecy.

IV. PRELIMINARY RESULTS

A. Test Bed and Correctness

Correctness, deletion, and interoperability tests run in an Ubuntu 24.04 QEMU 8.2 guest with kernel 6.8.0-147 and TCG, no KVM. The guest uses network namespaces `n1`/`n2` connected by a veth pair; a third namespace `n3` is used for the forwarded-path test. Throughput is measured separately on the native WSL2 kernel 6.18.54.1 (Hyper-V, 16 vCPUs), because TCG CPU emulation is not a throughput measurement. The native host is a VM, not bare metal. The direct-veth baseline is about 100 Gbit/s, so the link is not the bottleneck; the tunnel is CPU-bound.

Each run starts from a clean module, configures keys and peers, and captures outer packets. A host-side Python verifier independently reconstructs the nonce and AAD and decrypts each frame with the `cryptography` package. Counters are read from `/proc/torshield/stats`. Table I is the raw list. The 1 MB IPv4 and 256 KB IPv6 TCP transfers were byte-exact, and the independent verifier accepted every captured frame. The large-UDP cases exercise outer fragmentation and reassembly. Replay, tamper, and unknown-source packets were dropped with exact counter increments.

B. TC Fail-Closed Behaviour

In the `LOCAL_OUT` test, a local socket sends eight packets while the classifier is deleted. All eight fail with `EPERM`, the peer server receives zero, `tc_guard` increases by eight, and both `dev_out` and `tun_out` remain unchanged. Deleting the clsact qdisc produces the same result. Reattaching the classifier restores a 16/16 echo; detaching falls back to the Netfilter path at 8/8.

The `FORWARD` test injects raw frames on `n1`, so the sender's `LOCAL_OUT` hook is bypassed. With no classifier, `n2` receives eight forwarded inner packets, `n3` sees zero outer packets, `tc_guard` increases by eight, and `dev_out` does not move. With the classifier present, `tc_guard` stays flat, `dev_out` increases by eight, and `n3` records eight `tun_rx`. Deleting the classifier returns to zero outer packets and a guard increase of eight. That is the empirical basis for the fail-closed claim, with the residual race noted in Section III-B.

C. Throughput

On the native VM, the four-stream baseline was 100.98/99.76/97.84 Gbit/s. The tunnel reached 909.7/921.5/965.4 Mbit/s for one stream and 3345.6/3018.1/3203.1 Mbit/s for four streams. A 30 s four-stream run sustained 3.40 Gbit/s with `SUnreclaim` flat at 63–68 MB and zero drops or replays. The RTT average was 0.052 ms over 100 pings.

V. SECURITY ARGUMENT

The AEAD gives confidentiality and integrity for the inner IP packet, including its addresses and transport headers. The 64-bit per-peer sequence number and the 1024-packet window reject replays after authentication. The key identifier selects a per-peer key; the inner source ACL stops a peer from injecting a source prefix it does not own. Endpoint roaming happens only after authentication, so an off-path attacker cannot redirect a peer by spoofing an outer source. In tunnel and hybrid modes, every authentication and parsing failure drops the packet. In TC mode, the guard drops peer-destined packets when the classifier is missing, and the classifier itself drops a matching packet when its redirect target is unset. Padding to 64-byte buckets reduces exact-length leakage.

The limits are just as important. Outer addresses, ports, the `TS` magic, key identifier, sequence number, ciphertext length, direction, timing, and approximate size bucket are visible. The system does not resist traffic analysis or DPI. There is no handshake, no forward secrecy, and no automatic key agreement; a compromised key exposes the traffic it protects, and a compromised endpoint exposes plaintext. The peer table, key, and mode are global across network namespaces. The rate limiter is a fixed-window filter, not a cookie challenge, and the source is spoofable. The TC guard has a narrow check-to-egress race; the deletion tests prove fail-closed behavior for subsequent packets, not absolute zero-race closure. TC and hybrid mode are prototypes, not production-hardened transports.

On its own, TORSHIELD does not provide anonymity: it has no onion routing, cover traffic, or relay network. The next enhancement is to plug the outer tunnel into a Tor pluggable transport, so that the system can borrow Tor's anonymity and censorship resistance instead of claiming them. We have not built that integration yet, and it is separate from the current datapath.

VI. DISCUSSION AND NEXT STEPS

For current deployments, use `tunnel` mode. It fails closed on authentication and parsing errors and does not depend on a separately attached classifier. TC mode is experimental: after any peer add, delete, or clear, rerun `torshield-tc attach` before returning to `tc`, because the kernel forces the mode back to `tunnel`. TC mode should not be used where

a clear fallback is unacceptable until the guard-to-egress race is closed.

The next version starts with that race and with the handshake. Both are prerequisites for using TC mode outside a lab. The Tor pluggable-transport integration comes after them, because it changes the outer transport but not the inner policy.

Every number in this abstract comes from a script in the artifact repository. The QEMU harness starts from a clean module, writes keys and peer prefixes through `/proc`, captures on `n3`, and checks byte counts and counters. The native harness runs `iperf3` over veth. We keep the raw logs next to the scripts so that a rerun can be compared line by line.

VII. Doctoral Research Plan

A. Research Questions

The dissertation is organized around three questions:

- **RQ1 (correctness under transparency):** Can a transparent kernel tunnel preserve byte-exact IPv4/IPv6 semantics across GSO, fragmentation, reconnect, and peer change while remaining fail-closed when the TC classifier is removed?
- **RQ2 (cost of in-kernel protection):** What are the throughput and CPU costs of moving AEAD and per-peer policy into the kernel, compared with WireGuard and IPsec under identical traffic and hardware?
- **RQ3 (secure control plane):** Can a handshake with forward secrecy and a per-network-namespace control plane be added without reopening the steering race or weakening transparency?

B. Timeline and Milestones

The first year closes the guard-to-egress race with a last-chance transmit check or a single authoritative steering point, specifies a Noise-style or ephemeral X25519 handshake with a pre-shared key, and moves state to netlink and per-network-namespace storage. It also adds IPv6 and TCX deletion tests. The second year repeats the throughput, CPU, and robustness measurements on bare metal and a physical NIC, compares against WireGuard and IPsec under the same traffic, performs an external security review, and evaluates the Tor pluggable-transport integration as an outer transport wrapper. The third year packages the artifact for reproducibility, completes the dissertation, and prepares the defense.

The next evaluation will keep the same deletion matrix and add IPv6 and TCX cases. We will repeat the 30 s run on a physical NIC, report CPU time per packet and tail latency, and compare against WireGuard and IPsec on the same veth pair and with the same `iperf3` settings. That gives a data-path comparison under one harness, which the current aggregate throughput does not.

C. Expected Contributions

The expected contributions are (i) a transparent, fail-closed kernel tunnel with measured classifier-loss behavior; (ii) an open-source artifact and test harness for IPv4/IPv6, GSO, fragmentation, replay, and rekey; (iii) a cost analysis against WireGuard and IPsec that separates data-path from protocol cost; and (iv) a measured evaluation of the Tor pluggable-transport integration. These results address whether transparent kernel tunneling can be deployable and fail-closed at acceptable throughput.

VIII. Conclusion

TorShield is a transparent in-kernel authenticated tunnel with three steering paths and a kernel-side fail-closed guard for TC. The current code is byte-exact on IPv4 and IPv6 TCP, survives independent AEAD verification, drops replay/tamper/unknown-source packets, and blocks subsequent local and forwarded packets when the TC classifier is removed. On a native VM it sustains 3.40 Gbit/s over four streams for 30 s with flat unreclaimable memory. The next step is to integrate the outer tunnel with a Tor pluggable transport, which would add anonymity without changing the datapath. Until the handshake and race are addressed, TorShield is a research and teaching artifact, not a production replacement for WireGuard or IPsec.

References

[1] R. Dingledine, N. Mathewson, and P. Syverson, "Tor: The second-generation onion router," in *Proceedings of the 13th USENIX Security Symposium*, 2004, pp. 303–320.

[2] The Tor Project, "obfs4: A pluggable transport," https://gitlab.com/yawning/obfs4, 2024, accessed 2026-10-22.

[3] P. Winter, T. Pulls, and J. Fuss, "Scramblesuit: A polymorphic network protocol to circumvent censorship," in *Proceedings of the 12th ACM Workshop on Privacy in the Electronic Society*, 2013, pp. 213–224.

[4] K. P. Dyer, S. E. Coull, T. Ristenpart, and T. Shrimpton, "Protocol misidentification made easy with format-transforming encryption," in *Proceedings of the 2013 ACM SIGSAC Conference on Computer and Communications Security*, 2013, pp. 61–72.

[5] C. V. Wright, S. E. Coull, and F. Monrose, "Traffic morphing: An efficient defense against statistical traffic analysis," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2009.

[6] A. Houmansadr, C. Brubaker, and V. Shmatikov, "The parrot is dead: Observing unobservable network communications," in *Proceedings of the 2013 IEEE Symposium on Security and Privacy*, 2013, pp. 65–79.

[7] S. Kent and K. Seo, "Security architecture for the internet protocol," RFC 4301, IETF, 2005.

[8] J. A. Donenfeld, "Wireguard: Next generation kernel network tunnel," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2017.

[9] T. Høiland-Jørgensen, J. D. Brouer, D. Borkmann, J. Fastabend, T. Herbert, D. Ahern, and D. Miller, "The express data path: Fast programmable packet processing in the operating system kernel," in *Proceedings of the 14th International Conference on Emerging Networking Experiments and Technologies (CoNEXT)*, 2018, pp. 54–66.

[10] Y. Nir and A. Langley, "Chacha20 and poly1305 for ietf protocols," RFC 8439, IETF, 2018.